Gradient Descent — Notes

1. What is Gradient Descent?

Gradient Descent is an **iterative optimization algorithm** used to minimize a **loss/cost function**.

It is widely used in:

- Linear Regression
- Logistic Regression
- Neural Networks / Deep Learning

Goal: Find the parameter values that give the **minimum loss**.

---

2. Intuition

Imagine a person standing on a mountain and trying to reach the **lowest point of a valley**.

● Starting point

/

/

/ ↓

/ ● Minimum

At every step:

1. Calculate the **gradient (slope)**.
2. Gradient tells us the direction of **steepest increase**.
3. Move in the **opposite direction** to decrease the loss.
4. Repeat until reaching the minimum.

---

3. Learning Rate (η)

The **learning rate** controls how large each step is.

Small learning rate → slow convergence

Good learning rate → reaches minimum efficiently

Large learning rate → may overshoot/diverge

Example:

η = 0.01 → small steps

η = 0.1 → larger steps

η = 1.0 → potentially very large steps

---

4. Parameter Update

The general update rule is:

Where:

- **θ** → model parameter/weight
- **J** → loss function
- **η** → learning rate
- **∂J/∂θ** → gradient of the loss

The **minus sign** is important because the gradient points toward increasing loss, so we move in the opposite direction.

---

5. Example

Suppose:

Current weight = 5

Gradient = 2

Learning rate = 0.1

Then:

So Gradient Descent changes the weight from **5 → 4.8** to reduce the loss.

---

6. Loss Function Shape

Convex function:

- Has one clear global minimum.
- Gradient Descent can reach that minimum.

Non-convex function:

- Can contain local minima and saddle points.
- Finding the global minimum can be more difficult.

---

7. Importance of Feature Scaling

If features have very different scales, Gradient Descent can converge slowly.

Example:

Age → 20–60

Salary → 20,000–500,000

Scaling makes optimization more balanced and stable.

Batch Gradient Descent — Notes

1. Types of Gradient Descent

Gradient Descent updates model parameters to **minimize the loss**.

There are 3 main types:

Batch Gradient Descent (BGD)

- Uses **the entire training dataset** to calculate the gradient.
- Gives stable and accurate updates.
- Can be slow for very large datasets.

Stochastic Gradient Descent (SGD)

- Uses **one training row at a time**.
- Much faster for large datasets.
- Updates are noisy and less stable.

Mini-Batch Gradient Descent

- Uses a **small group of rows** at a time.
- Combines the advantages of BGD and SGD.
- Commonly used in Deep Learning.

BGD → All rows

SGD → 1 row

Mini-Batch → Few rows

---

2. Batch Gradient Descent in Multiple Linear Regression

Suppose:

X1 = Age

X2 = Experience

X3 = Education

↓

Model

↓

y

The model is:

There are multiple parameters:

b → intercept

w1 → coefficient for X1

w2 → coefficient for X2

w3 → coefficient for X3

BGD calculates the error using **all training rows**, then updates **all parameters**.

---

3. Parameter Update

For every iteration/epoch:

1. Make predictions

↓

2. Calculate errors

↓

3. Calculate gradients

↓

4. Update coefficients & intercept

↓

5. Repeat

The general idea is:

The gradient tells us **which direction to move**, while the learning rate controls **how far to move**.

---

4. Why "Batch"?

Suppose you have **353 training rows**.

With Batch Gradient Descent:

353 rows

↓

Calculate gradient using ALL 353

↓

Update weights

↓

Next epoch

It does **not** update after every individual row.

---

5. Epoch

An **epoch** means the model has gone through the **entire training dataset once**.

Example:

1000 rows

10 epochs

means the algorithm processes those 1000 training rows **10 times**.

More epochs generally give the model more opportunities to reach the minimum, but too many can waste computation.

---

6. Coding Concept

The GDRegressor class usually starts with:

self.coef_ = np.zeros(X.shape[1])

self.intercept_ = 0

Then repeatedly:

y_pred = np.dot(X, self.coef_) + self.intercept_

error = y - y_pred

self.coef_ = self.coef_ + learning_rate * gradient

self.intercept_ = self.intercept_ + learning_rate * gradient